

Lab 1

Importing packages and data

```
In [443... !pip3 install pandas seaborn numpy
import numpy as np
import pandas as pd
import seaborn as sns
```

```
Requirement already satisfied: pandas in /Library/Frameworks/Python.framework/Versions/3.13/lib/python3.13/site-packages (2.3.1)
Requirement already satisfied: seaborn in /Library/Frameworks/Python.framework/Versions/3.13/lib/python3.13/site-packages (0.13.2)
Requirement already satisfied: numpy in /Library/Frameworks/Python.framework/Versions/3.13/lib/python3.13/site-packages (2.3.1)
Requirement already satisfied: python-dateutil>=2.8.2 in /Library/Frameworks/Python.framework/Versions/3.13/lib/python3.13/site-packages (from pandas) (2.9.0.post0)
Requirement already satisfied: pytz>=2020.1 in /Library/Frameworks/Python.framework/Versions/3.13/lib/python3.13/site-packages (from pandas) (2025.2)
Requirement already satisfied: tzdata>=2022.7 in /Library/Frameworks/Python.framework/Versions/3.13/lib/python3.13/site-packages (from pandas) (2025.2)
Requirement already satisfied: matplotlib!=3.6.1,>=3.4 in /Library/Frameworks/Python.framework/Versions/3.13/lib/python3.13/site-packages (from seaborn) (3.10.3)
Requirement already satisfied: contourpy>=1.0.1 in /Library/Frameworks/Python.framework/Versions/3.13/lib/python3.13/site-packages (from matplotlib!=3.6.1,>=3.4->seaborn) (1.3.3)
Requirement already satisfied: cycler>=0.10 in /Library/Frameworks/Python.framework/Versions/3.13/lib/python3.13/site-packages (from matplotlib!=3.6.1,>=3.4->seaborn) (0.12.1)
Requirement already satisfied: fonttools>=4.22.0 in /Library/Frameworks/Python.framework/Versions/3.13/lib/python3.13/site-packages (from matplotlib!=3.6.1,>=3.4->seaborn) (4.59.0)
Requirement already satisfied: kiwisolver>=1.3.1 in /Library/Frameworks/Python.framework/Versions/3.13/lib/python3.13/site-packages (from matplotlib!=3.6.1,>=3.4->seaborn) (1.4.8)
Requirement already satisfied: packaging>=20.0 in /Library/Frameworks/Python.framework/Versions/3.13/lib/python3.13/site-packages (from matplotlib!=3.6.1,>=3.4->seaborn) (25.0)
Requirement already satisfied: pillow>=8 in /Library/Frameworks/Python.framework/Versions/3.13/lib/python3.13/site-packages (from matplotlib!=3.6.1,>=3.4->seaborn) (11.3.0)
Requirement already satisfied: pyparsing>=2.3.1 in /Library/Frameworks/Python.framework/Versions/3.13/lib/python3.13/site-packages (from matplotlib!=3.6.1,>=3.4->seaborn) (3.2.3)
Requirement already satisfied: six>=1.5 in /Library/Frameworks/Python.framework/Versions/3.13/lib/python3.13/site-packages (from python-dateutil>=2.8.2->pandas) (1.17.0)
```

[notice] A new release of pip is available: 25.1.1 -> 26.1.2

[notice] To update, run: pip3 install --upgrade pip

Task 1

```
In [444... raw_city = pd.read_csv("data/city_day.csv")
raw_crop = pd.read_csv("data/crop_production.csv")
```

Getting a feel for the data by printing first 5 rows.

```
In [445... raw_city.head()
```

Out [445...

	City	Date	PM2.5	PM10	NO	NO2	NOx	NH3	CO	SO2	O3
0	Ahmedabad	2015-01-01	NaN	NaN	0.92	18.22	17.15	NaN	0.92	27.64	133.30
1	Ahmedabad	2015-01-02	NaN	NaN	0.97	15.69	16.46	NaN	0.97	24.55	34.00
2	Ahmedabad	2015-01-03	NaN	NaN	17.40	19.30	29.70	NaN	17.40	29.07	30.70
3	Ahmedabad	2015-01-04	NaN	NaN	1.70	18.48	17.97	NaN	1.70	18.59	36.00
4	Ahmedabad	2015-01-05	NaN	NaN	22.10	21.42	37.76	NaN	22.10	39.33	39.30

```
In [446... raw_crop.head()
```

Out [446...

	State_Name	District_Name	Crop_Year	Season	Crop	Area	Production
0	Andaman and Nicobar Islands	NICOBARS	2000	Kharif	Arecanut	1254.0	2000.0
1	Andaman and Nicobar Islands	NICOBARS	2000	Kharif	Other Kharif pulses	2.0	1.0
2	Andaman and Nicobar Islands	NICOBARS	2000	Kharif	Rice	102.0	321.0
3	Andaman and Nicobar Islands	NICOBARS	2000	Whole Year	Banana	176.0	641.0
4	Andaman and Nicobar Islands	NICOBARS	2000	Whole Year	Cashewnut	720.0	165.0

```
In [447... raw_city.shape
```

Out[447... (29531, 16)

In [448... `raw_crop.shape`

Out[448... (246091, 7)

Finding the column data types.

In [449... `raw_city.info()`

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 29531 entries, 0 to 29530
Data columns (total 16 columns):
#   Column          Non-Null Count  Dtype
---  -
0   City             29531 non-null  object
1   Date             29531 non-null  object
2   PM2.5            24933 non-null  float64
3   PM10             18391 non-null  float64
4   NO               25949 non-null  float64
5   NO2              25946 non-null  float64
6   NOx              25346 non-null  float64
7   NH3              19203 non-null  float64
8   CO               27472 non-null  float64
9   SO2              25677 non-null  float64
10  O3               25509 non-null  float64
11  Benzene          23908 non-null  float64
12  Toluene          21490 non-null  float64
13  Xylene           11422 non-null  float64
14  AQI              24850 non-null  float64
15  AQI_Bucket       24850 non-null  object
dtypes: float64(13), object(3)
memory usage: 3.6+ MB
```

In [450... `raw_crop.info()`

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 246091 entries, 0 to 246090
Data columns (total 7 columns):
#   Column          Non-Null Count  Dtype
---  -
0   State_Name       246091 non-null  object
1   District_Name    246091 non-null  object
2   Crop_Year        246091 non-null  int64
3   Season           246091 non-null  object
4   Crop             246091 non-null  object
5   Area             246091 non-null  float64
6   Production        242361 non-null  float64
dtypes: float64(2), int64(1), object(4)
memory usage: 13.1+ MB
```

Finding statistical mean, standard deviation, median and quartiles, max and min.

In [451... `raw_city.describe()`

Out [451...

	PM2.5	PM10	NO	NO2	NOx
count	24933.000000	18391.000000	25949.000000	25946.000000	25346.000000
mean	67.450578	118.127103	17.574730	28.560659	32.309123
std	64.661449	90.605110	22.785846	24.474746	31.646011
min	0.040000	0.010000	0.020000	0.010000	0.000000
25%	28.820000	56.255000	5.630000	11.750000	12.820000
50%	48.570000	95.680000	9.890000	21.690000	23.520000
75%	80.590000	149.745000	19.950000	37.620000	40.127500
max	949.990000	1000.000000	390.680000	362.210000	467.630000

In [452...

raw_crop.describe()

Out [452...

	Crop_Year	Area	Production
count	246091.000000	2.460910e+05	2.423610e+05
mean	2005.643018	1.200282e+04	5.825034e+05
std	4.952164	5.052340e+04	1.706581e+07
min	1997.000000	4.000000e-02	0.000000e+00
25%	2002.000000	8.000000e+01	8.800000e+01
50%	2006.000000	5.820000e+02	7.290000e+02
75%	2010.000000	4.392000e+03	7.023000e+03
max	2015.000000	8.580100e+06	1.250800e+09

Finding the number of missing values in each column of the dataset, if any.

In [453...

raw_city.isnull().sum()

Out [453...

```

City          0
Date          0
PM2.5        4598
PM10        11140
NO           3582
NO2          3585
NOx          4185
NH3         10328
CO           2059
SO2          3854
O3           4022
Benzene       5623
Toluene       8041
Xylene       18109
AQI           4681
AQI_Bucket    4681
dtype: int64

```

```
In [454... raw_crop.isnull().sum()
```

```
Out[454... State_Name      0
District_Name  0
Crop_Year      0
Season         0
Crop           0
Area           0
Production     3730
dtype: int64
```

Checking for duplicate rows.

```
In [455... raw_city.duplicated().sum()
```

```
Out[455... np.int64(0)
```

```
In [456... raw_crop.duplicated().sum()
```

```
Out[456... np.int64(0)
```

Checking unique values in each column.

```
In [457... raw_city.nunique()
```

```
Out[457... City           26
Date           2009
PM2.5          11716
PM10           12571
NO             5776
NO2            7404
NOx            8156
NH3            5922
CO             1779
SO2            4761
O3             7699
Benzene        1873
Toluene        3608
Xylene         1561
AQI            829
AQI_Bucket      6
dtype: int64
```

```
In [458... raw_crop.nunique()
```

```
Out[458... State_Name      33
District_Name  646
Crop_Year      19
Season         6
Crop           124
Area           38442
Production     51627
dtype: int64
```

Data Profile: city_day.csv

Property	Value
Rows × Columns	29,531 × 16
Numeric columns	13 (PM2.5, PM10, NO, NO2, NOx, NH3, CO, SO2, O3, Benzene, Toluene, Xylene, AQI — all float64)
Categorical columns	3 (City, Date, AQI_Bucket — all object)
Date range	2015-01-01 to 2020-07-01
Cities	26
AQI Buckets	6
Duplicate rows	0
Memory usage	~3.6 MB

Missing values:

Column	Missing	% Missing
Xylene	18,109	61.3%
PM10	11,140	37.7%
NH3	10,328	35.0%
Toluene	8,041	27.2%
Benzene	5,623	19.0%
AQI / AQI_Bucket	4,681	15.9%
PM2.5	4,598	15.6%
NOx	4,185	14.2%
O3	4,022	13.6%
SO2	3,854	13.1%
NO / NO2	~3,583	~12.1%
CO	2,059	7.0%

Data Profile: crop_production.csv

Property	Value
Rows × Columns	246,091 × 7
Numeric columns	3 (Crop_Year int64, Area float64, Production float64)
Categorical columns	4 (State_Name, District_Name, Season, Crop — all object)
Year range	1997–2015
States	33
Districts	646
Crops	124

Property		Value
Seasons	6	
Duplicate rows	0	
Memory usage	~13.1 MB	

Missing values:

Column	Missing	% Missing
Production	3,730	1.5%
All others	0	0%

Observations and Concerns:

- **city_day.csv** has heavy missing data across most pollutant columns. Xylene is 61% missing and is likely unusable as a feature without heavy imputation. PM10 (37.7%) and NH3 (35%) are also problematic.
- The `Date` column is stored as `object` instead of `datetime`, which needs conversion before any time-based analysis.
- **crop_production.csv** has extreme outliers in Production — max is 1.25 billion against a median of 729, suggesting the column may mix different units or scales across crops.
- Production has 3,730 missing values (1.5%). Since Production is likely the target variable for modelling, rows with missing Production may need to be dropped.
- No units are documented for Area or Production, making it unclear what is being measured.

Task 2

Creating working copies to preserve raw data.

```
In [459... city = raw_city.copy()
crop = raw_crop.copy()
```

Calculating the percentage of missing values in each column.

```
In [460... city_missing = (city.isnull().sum() / len(city) * 100).round(1)
city_missing = city_missing[city_missing > 0].sort_values(ascending=False)
city_missing
```

```
Out [460...] Xylene          61.3
              PM10         37.7
              NH3          35.0
              Toluene       27.2
              Benzene       19.0
              AQI           15.9
              AQI_Bucket    15.9
              PM2.5         15.6
              NOx           14.2
              O3            13.6
              SO2           13.1
              NO            12.1
              NO2           12.1
              CO             7.0
              dtype: float64
```

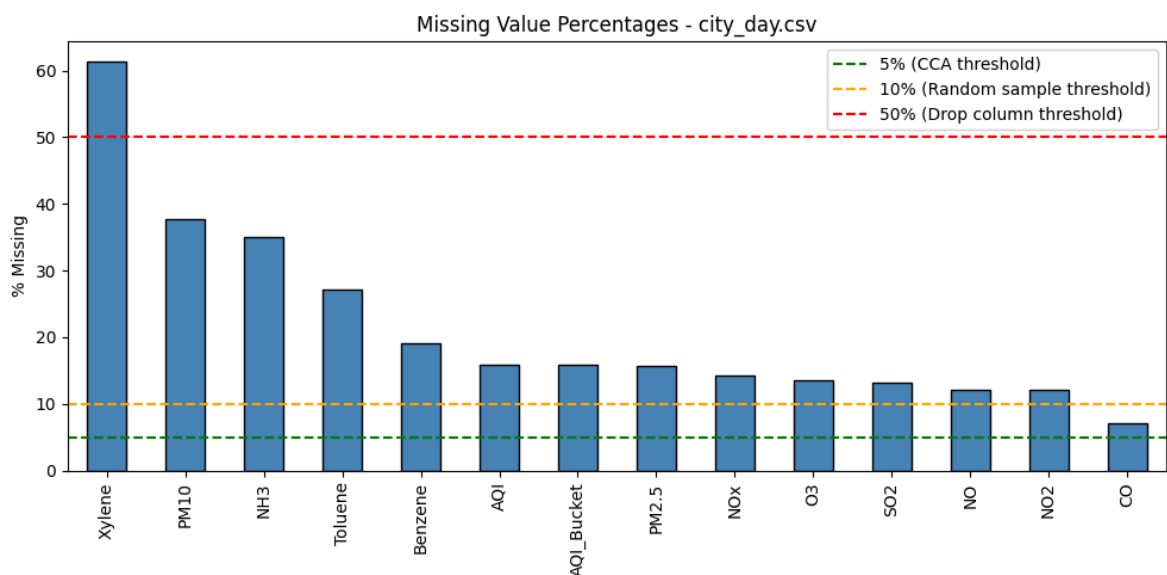
```
In [461...] crop_missing = (crop.isnull().sum() / len(crop) * 100).round(1)
              crop_missing = crop_missing[crop_missing > 0].sort_values(ascending=False)
              crop_missing
```

```
Out [461...] Production    1.5
              dtype: float64
```

Plotting missing value percentages with strategy thresholds.

```
In [462...] import matplotlib.pyplot as plt

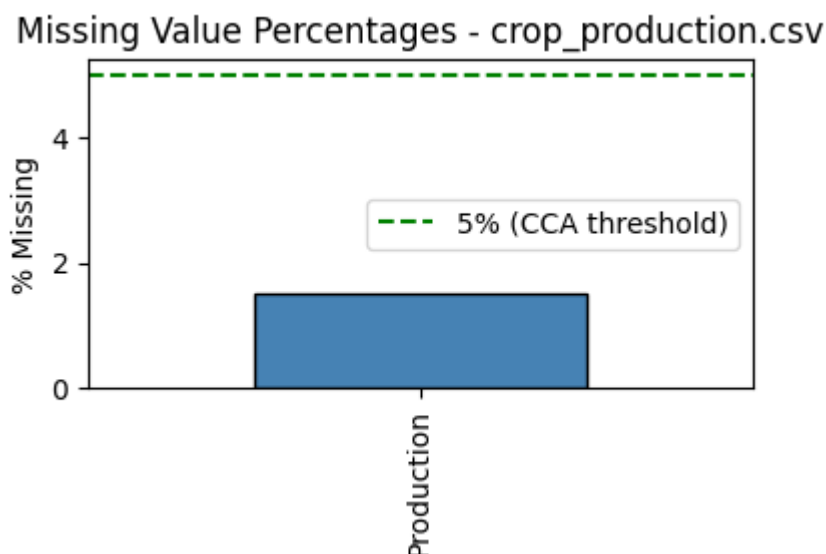
fig, ax = plt.subplots(figsize=(10, 5))
city_missing.plot(kind='bar', ax=ax, color='steelblue', edgecolor='black')
ax.axhline(y=5, color='green', linestyle='--', label='5% (CCA threshold)')
ax.axhline(y=10, color='orange', linestyle='--', label='10% (Random sample threshold)')
ax.axhline(y=50, color='red', linestyle='--', label='50% (Drop column threshold)')
ax.set_ylabel('% Missing')
ax.set_title('Missing Value Percentages - city_day.csv')
ax.legend()
plt.tight_layout()
plt.show()
```



```
In [463...] fig, ax = plt.subplots(figsize=(4, 3))
              crop_missing.plot(kind='bar', ax=ax, color='steelblue', edgecolor='black')
              ax.axhline(y=5, color='green', linestyle='--', label='5% (CCA threshold)')
```



```
ax.set_ylabel('% Missing')
ax.set_title('Missing Value Percentages - crop_production.csv')
ax.legend()
plt.tight_layout()
plt.show()
```



Missing Value Treatment Strategy

Approach based on percentage of missing values:

- **< 5%** - Complete Case Analysis (drop affected rows), minimal data loss
- **5-10%** - Median imputation (preferred over mean for skewed data, as mean is sensitive to outliers)
- **10-50%** - Random sample imputation (preserves original distribution shape)
- **> 50%** - Drop the column (more data is missing than present, imputation unreliable)

crop_production.csv

Column	% Missing	Decision	Reason
Production	1.5%	Drop rows (CCA)	Below 5%. Likely the target variable - imputing it would introduce noise in the outcome

city_day.csv

Column	% Missing	Decision	Reason
Xylene	61.3%	Drop column	Over 50% missing - any imputation would fabricate more data than it preserves
PM10	37.7%	Random sample	10-50% range - random sampling preserves distribution without shifting mean
NH3	35.0%	Random sample	Same as above

Column	% Missing	Decision	Reason
Toluene	27.2%	Random sample	Same as above
Benzene	19.0%	Random sample	Same as above
AQI	15.9%	Random sample	Same as above
AQI_Bucket	15.9%	Random sample	Categorical - random sampling maintains class proportions
PM2.5	15.6%	Random sample	Same as above
NOx	14.2%	Random sample	Same as above
O3	13.6%	Random sample	Same as above
SO2	13.1%	Random sample	Same as above
NO	12.1%	Random sample	Same as above
NO2	12.1%	Random sample	Same as above
CO	7.0%	Median imputation	5-10% range. CO is right-skewed (mean=2.25 vs median=0.89) - median is more robust than mean

Recording null counts before treatment.

```
In [464... before_city = city.isnull().sum()
before_crop = crop.isnull().sum()
```

Applying CCA to crop_production - dropping rows where Production is null (1.5%).

```
In [465... crop.dropna(subset=['Production'], inplace=True)
n_dropped = before_crop['Production']
print(f"Dropped {n_dropped} rows. New shape: {crop.shape}")
```

Dropped 3730 rows. New shape: (242361, 7)

Dropping Xylene column from city_day - 61.3% missing, imputation unreliable.

```
In [466... city.drop(columns=['Xylene'], inplace=True)
print(f"Dropped Xylene column. New shape: {city.shape}")
```

Dropped Xylene column. New shape: (29531, 15)

Applying median imputation to CO (7.0%). Using median over mean as CO is right-skewed.

```
In [467... fig, axes = plt.subplots(1, 2, figsize=(12, 4))
```

```

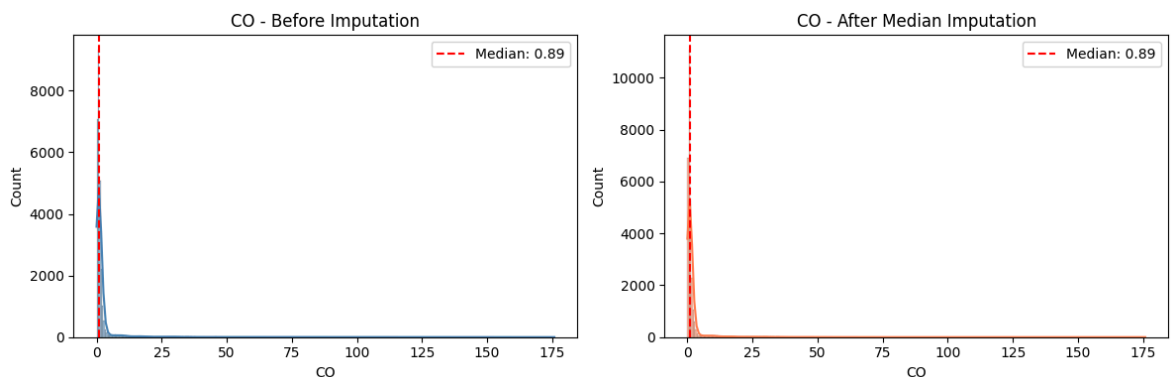
median_co = city['CO'].median()
sns.histplot(city['CO'].dropna(), kde=True, ax=axes[0], color='steelblue')
axes[0].axvline(median_co, color='red', linestyle='--', label=f'Median: {median_co}')
axes[0].set_title('CO - Before Imputation')
axes[0].legend()

city['CO'] = city['CO'].fillna(median_co)

sns.histplot(city['CO'], kde=True, ax=axes[1], color='coral')
axes[1].axvline(median_co, color='red', linestyle='--', label=f'Median: {median_co}')
axes[1].set_title('CO - After Median Imputation')
axes[1].legend()

plt.tight_layout()
plt.show()

```



Applying random sample imputation to remaining columns (>10% missing).

```

In [468... def random_sample_impute(df, col):
    non_null = df[col].dropna()
    null_idx = df[df[col].isnull()].index
    samples = non_null.sample(len(null_idx), replace=True, random_state=4)
    samples.index = null_idx
    df.loc[null_idx, col] = samples

cols_to_impute = list(city.columns[city.isnull().any()])
before_dist = {col: city[col].dropna().copy() for col in cols_to_impute}

for col in cols_to_impute:
    random_sample_impute(city, col)

print(f"Imputed {len(cols_to_impute)} columns: {cols_to_impute}")

```

Imputed 12 columns: ['PM2.5', 'PM10', 'NO', 'NO2', 'NOx', 'NH3', 'SO2', 'O3', 'Benzene', 'Toluene', 'AQI', 'AQI_Bucket']

Plotting distributions before and after random sample imputation for numeric columns.

```

In [469... numeric_cols = [c for c in cols_to_impute if city[c].dtype != 'object']
n = len(numeric_cols)
nrows = (n + 2) // 3

fig, axes = plt.subplots(nrows, 3, figsize=(15, 4 * nrows))
axes = axes.flatten()

for i, col in enumerate(numeric_cols):

```

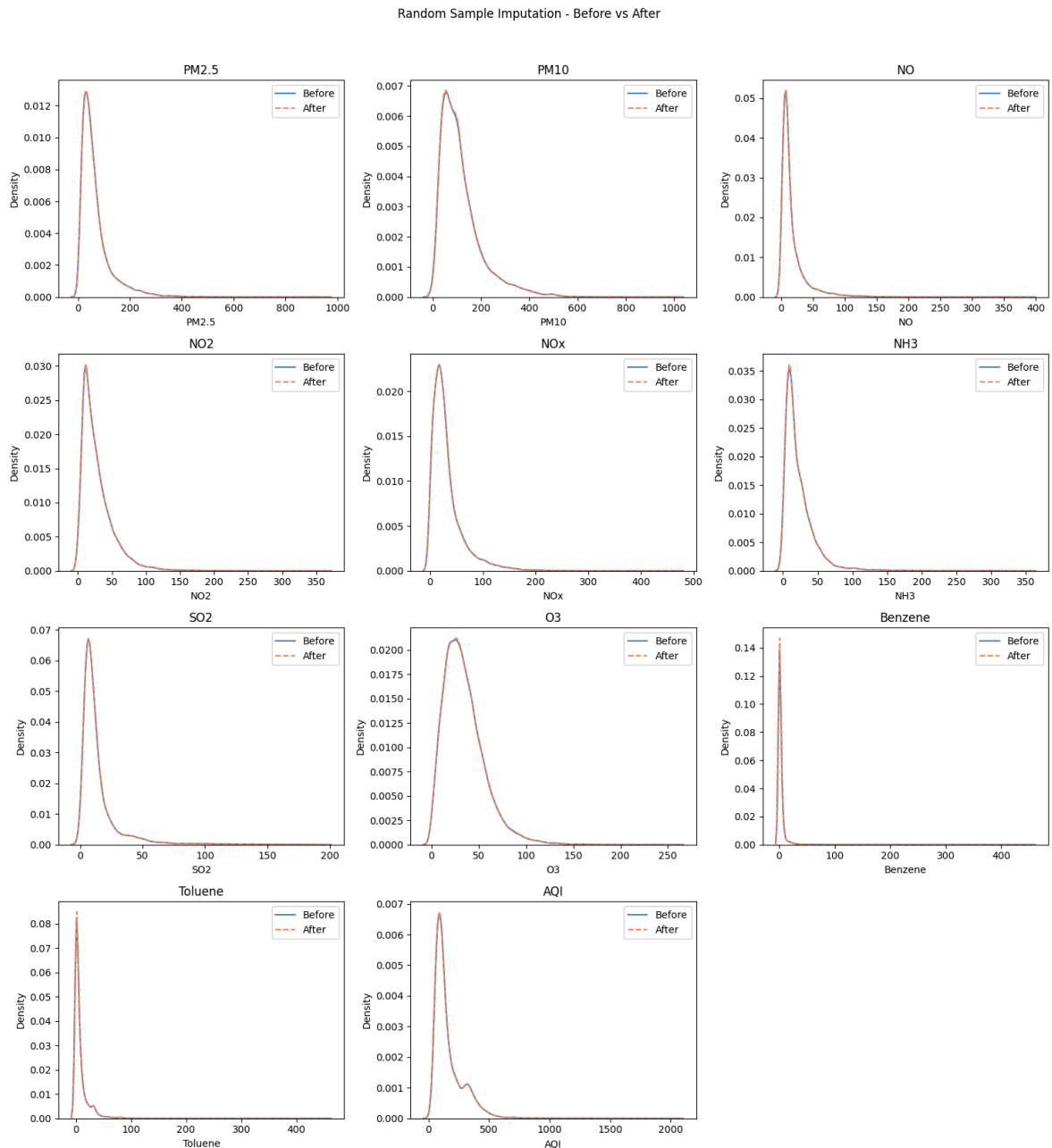
```

sns.kdeplot(before_dist[col], ax=axes[i], label='Before', color='steelblue')
sns.kdeplot(city[col], ax=axes[i], label='After', color='coral', line
axes[i].set_title(col)
axes[i].legend()

for j in range(len(numeric_cols), len(axes)):
    axes[j].set_visible(False)

plt.suptitle('Random Sample Imputation - Before vs After', y=1.02)
plt.tight_layout()
plt.show()

```

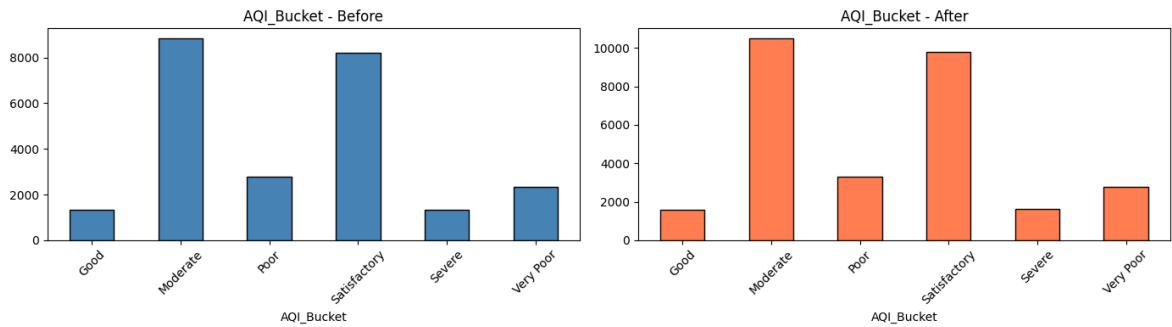


```

In [470]: if 'AQI_Bucket' in cols_to_impute:
fig, axes = plt.subplots(1, 2, figsize=(14, 4))
before_dist['AQI_Bucket'].value_counts().sort_index().plot(kind='bar')
axes[0].set_title('AQI_Bucket - Before')
axes[0].tick_params(axis='x', rotation=45)
city['AQI_Bucket'].value_counts().sort_index().plot(kind='bar', ax=axes[1])
axes[1].set_title('AQI_Bucket - After')
axes[1].tick_params(axis='x', rotation=45)

```

```
plt.tight_layout()
plt.show()
```



Verifying null counts after treatment.

```
In [471... after_city = city.isnull().sum()
after_crop = crop.isnull().sum()

print("=== crop_production - Before vs After ===")
print(f"Production: {before_crop['Production']} -> {after_crop['Production']}")
print(f"Rows: {len(raw_crop)} -> {len(crop)}")
print()

print("=== city_day - Before vs After ===")
for col in before_city[before_city > 0].index:
    if col in after_city.index:
        print(f"{col}: {before_city[col]} -> {after_city[col]}")
    else:
        print(f"{col}: {before_city[col]} -> [column dropped]")

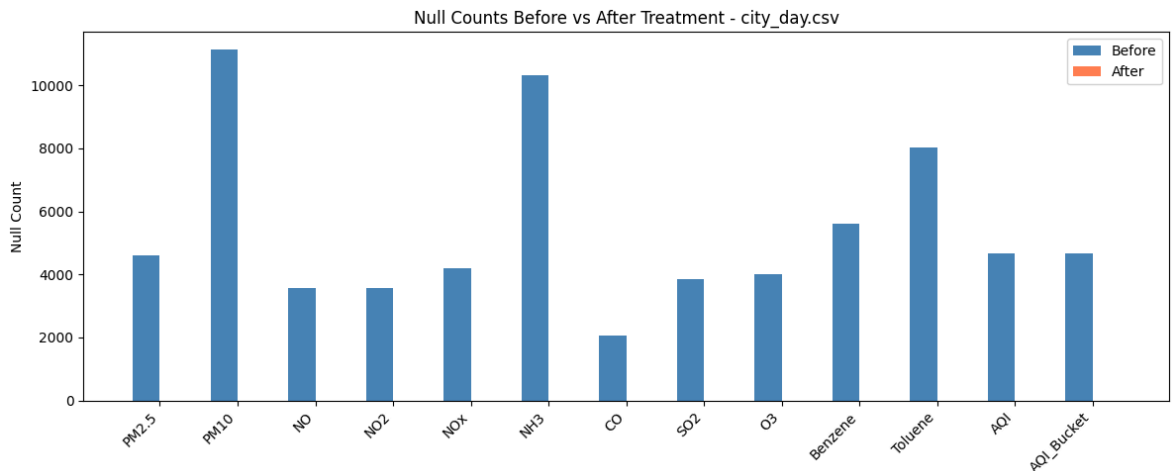
=== crop_production - Before vs After ===
Production: 3730 -> 0
Rows: 246091 -> 242361

=== city_day - Before vs After ===
PM2.5: 4598 -> 0
PM10: 11140 -> 0
NO: 3582 -> 0
NO2: 3585 -> 0
NOx: 4185 -> 0
NH3: 10328 -> 0
CO: 2059 -> 0
SO2: 3854 -> 0
O3: 4022 -> 0
Benzene: 5623 -> 0
Toluene: 8041 -> 0
Xylene: 18109 -> [column dropped]
AQI: 4681 -> 0
AQI_Bucket: 4681 -> 0
```

```
In [472... common_cols = [c for c in before_city[before_city > 0].index if c in after_city[after_city > 0].index]
before_vals = [before_city[c] for c in common_cols]
after_vals = [after_city[c] for c in common_cols]

fig, ax = plt.subplots(figsize=(12, 5))
x = range(len(common_cols))
width = 0.35
ax.bar([i - width/2 for i in x], before_vals, width, label='Before', color='blue')
ax.bar([i + width/2 for i in x], after_vals, width, label='After', color='orange')
ax.set_xticks(list(x))
```

```
ax.set_xticklabels(common_cols, rotation=45, ha='right')
ax.set_ylabel('Null Count')
ax.set_title('Null Counts Before vs After Treatment - city_day.csv')
ax.legend()
plt.tight_layout()
plt.show()
```



Task 3

Checking for inconsistencies in string columns that would break a merge on State.

Recording row counts before cleaning.

```
In [473... print(f"city: {city.shape}")
print(f"crop: {crop.shape}")
```

```
city: (29531, 15)
crop: (242361, 7)
```

Checking for leading/trailing whitespace in all string columns.

```
In [474... for col in city.select_dtypes(include='object').columns:
    n = (city[col].dropna() != city[col].dropna().str.strip()).sum()
    if n > 0:
        print(f"{col}: {n} values with extra spaces")
        print(f"  Examples: {city[city[col].dropna() != city[col].dropna().str.strip()][col].values}")
    else:
        print(f"{col}: clean")
```

```
City: clean
Date: clean
AQI_Bucket: clean
```

```
In [475... for col in crop.select_dtypes(include='object').columns:
    n = (crop[col] != crop[col].str.strip()).sum()
    if n > 0:
        print(f"{col}: {n} values with extra spaces")
        print(f"  Examples: {crop[crop[col] != crop[col].str.strip()][col].values}")
    else:
        print(f"{col}: clean")
```

State_Name: 7223 values with extra spaces
Examples: ['Jammu and Kashmir ' 'Telangana ']
District_Name: clean
Season: 242361 values with extra spaces
Examples: ['Kharif ' 'Whole Year ' 'Autumn ' 'Rabi ' 'Summer ']
Crop: 1958 values with extra spaces
Examples: ['Coconut ']

Listing unique values in State_Name and Season to spot spelling variants.

```
In [476... print("=== State_Name unique values (showing repr to expose spaces) ===")
for s in sorted(crop['State_Name'].unique()):
    print(repr(s))
```

```
=== State_Name unique values (showing repr to expose spaces) ===  
'Andaman and Nicobar Islands'  
'Andhra Pradesh'  
'Arunachal Pradesh'  
'Assam'  
'Bihar'  
'Chandigarh'  
'Chhattisgarh'  
'Dadra and Nagar Haveli'  
'Goa'  
'Gujarat'  
'Haryana'  
'Himachal Pradesh'  
'Jammu and Kashmir '  
'Jharkhand'  
'Karnataka'  
'Kerala'  
'Madhya Pradesh'  
'Maharashtra'  
'Manipur'  
'Meghalaya'  
'Mizoram'  
'Nagaland'  
'Odisha'  
'Puducherry'  
'Punjab'  
'Rajasthan'  
'Sikkim'  
'Tamil Nadu'  
'Telangana '  
'Tripura'  
'Uttar Pradesh'  
'Uttarakhand'  
'West Bengal'  
'Andaman and Nicobar Islands'  
'Andhra Pradesh'  
'Arunachal Pradesh'  
'Assam'  
'Bihar'  
'Chandigarh'  
'Chhattisgarh'  
'Dadra and Nagar Haveli'  
'Goa'  
'Gujarat'  
'Haryana'  
'Himachal Pradesh'  
'Jammu and Kashmir '  
'Jharkhand'  
'Karnataka'  
'Kerala'  
'Madhya Pradesh'  
'Maharashtra'  
'Manipur'  
'Meghalaya'  
'Mizoram'  
'Nagaland'  
'Odisha'  
'Puducherry'  
'Punjab'  
'Rajasthan'
```



```
'Sikkim'
'Tamil Nadu'
'Telangana '
'Tripura'
'Uttar Pradesh'
'Uttarakhand'
'West Bengal'
```

```
In [477... print("=== Season unique values ===")
for s in sorted(crop['Season'].unique()):
    print(repr(s))
```

```
=== Season unique values ===
'Autumn      '
'Kharif      '
'Rabi        '
'Summer      '
'Whole Year  '
'Winter      '
```

Checking if city_day can merge with crop_production on State. city_day has no State column, so we need to map cities to states.

```
In [478... city_to_state = {
    'Ahmedabad': 'Gujarat', 'Aizawl': 'Mizoram', 'Amaravati': 'Andhra Pra
    'Amritsar': 'Punjab', 'Bengaluru': 'Karnataka', 'Bhopal': 'Madhya Pra
    'Brajrajnagar': 'Odisha', 'Chandigarh': 'Chandigarh', 'Chennai': 'Tam
    'Coimbatore': 'Tamil Nadu', 'Delhi': 'Delhi', 'Ernakulam': 'Kerala',
    'Gurugram': 'Haryana', 'Guwahati': 'Assam', 'Hyderabad': 'Telangana',
    'Jaipur': 'Rajasthan', 'Jorapokhar': 'Jharkhand', 'Kochi': 'Kerala',
    'Kolkata': 'West Bengal', 'Lucknow': 'Uttar Pradesh', 'Mumbai': 'Maha
    'Patna': 'Bihar', 'Shillong': 'Meghalaya', 'Talcher': 'Odisha',
    'Thiruvananthapuram': 'Kerala', 'Visakhapatnam': 'Andhra Pradesh'
}

city['State'] = city['City'].map(city_to_state)
print("Added State column to city_day.")
print(city[['City', 'State']].drop_duplicates().sort_values('State').to_s
```

Added State column to city_day.

City	State
Visakhapatnam	Andhra Pradesh
Amaravati	Andhra Pradesh
Guwahati	Assam
Patna	Bihar
Chandigarh	Chandigarh
Delhi	Delhi
Ahmedabad	Gujarat
Gurugram	Haryana
Jorapokhar	Jharkhand
Bengaluru	Karnataka
Kochi	Kerala
Ernakulam	Kerala
Thiruvananthapuram	Kerala
Bhopal	Madhya Pradesh
Mumbai	Maharashtra
Shillong	Meghalaya
Aizawl	Mizoram
Brajrajnagar	Odisha
Talcher	Odisha
Amritsar	Punjab
Jaipur	Rajasthan
Chennai	Tamil Nadu
Coimbatore	Tamil Nadu
Hyderabad	Telangana
Lucknow	Uttar Pradesh
Kolkata	West Bengal

```
In [479... city_states = set(city['State'].unique())
crop_states_raw = set(crop['State_Name'].unique())
crop_states_clean = set(s.strip() for s in crop['State_Name'].unique())

print("States in city_day but NOT in crop_production:")
print(city_states - crop_states_clean)
print()
print("States in crop_production but NOT in city_day:")
print(crop_states_clean - city_states)
```

States in city_day but NOT in crop_production:
{'Delhi'}

States in crop_production but NOT in city_day:
{'Goa', 'Manipur', 'Nagaland', 'Dadra and Nagar Haveli', 'Uttarakhand', 'Puducherry', 'Jammu and Kashmir', 'Arunachal Pradesh', 'Andaman and Nicobar Islands', 'Tripura', 'Chhattisgarh', 'Himachal Pradesh', 'Sikkim'}

Inconsistencies Found

#	File	Column	Issue	Affected Rows	Fix
1	crop_production	State_Name	Trailing space on "Jammu and Kashmir "	3,478	Strip whitespace
2	crop_production	State_Name	Trailing space on "Telangana "	3,805	Strip whitespace

#	File	Column	Issue	Affected Rows	Fix
3	crop_production	Season	ALL values have trailing spaces ("Kharif ", etc.)	246,091	Strip whitespace
4	crop_production	Crop	Trailing space on "Coconut "	1,985	Strip whitespace
5	city_day	(none)	No State column exists for merging	29,531	Add State column via city-to-state mapping
6	city_day	State	"Delhi" has no match in crop_production	2,009	Noted - these rows will not join

Why this matters: A merge on State would silently drop all "Jammu and Kashmir " and "Telangana " rows (7,283 records) because the trailing space makes them different strings from their clean counterparts.

Applying fixes: stripping whitespace from all string columns in both datasets.

```
In [480... # Strip whitespace from all object columns in crop
for col in crop.select_dtypes(include='object').columns:
    crop[col] = crop[col].str.strip()

# Strip whitespace from all object columns in city
for col in city.select_dtypes(include='object').columns:
    city[col] = city[col].str.strip()

print("Stripped whitespace from all string columns.")
```

Stripped whitespace from all string columns.

Checking for duplicate rows after cleaning (spaces could have hidden duplicates).

```
In [481... print(f"city exact duplicates: {city.duplicated().sum()}")
print(f"crop exact duplicates: {crop.duplicated().sum()}")

# Also check logical duplicates on key columns
city_key_dups = city.duplicated(subset=['City', 'Date']).sum()
crop_key_dups = city_key_dups = crop.duplicated(subset=['State_Name', 'Di
print(f"city key duplicates (City+Date): {city.duplicated(subset=['City',
print(f"crop key duplicates (State+District+Year+Season+Crop): {crop_key_
```

```
city exact duplicates: 0
crop exact duplicates: 0
city key duplicates (City+Date): 0
crop key duplicates (State+District+Year+Season+Crop): 0
```

Verifying the fixes worked.

```
In [482... print("=== Remaining whitespace issues ===")
issues = 0
for col in crop.select_dtypes(include='object').columns:
    n = (crop[col] != crop[col].str.strip()).sum()
    if n > 0:
```

```

        print(f"crop {col}: {n}")
        issues += 1
    for col in city.select_dtypes(include='object').columns:
        n = (city[col].dropna() != city[col].dropna().str.strip()).sum()
        if n > 0:
            print(f"city {col}: {n}")
            issues += 1
    if issues == 0:
        print("None - all string columns are clean.")

```

=== Remaining whitespace issues ===
 None - all string columns are clean.

In [483... `print("=== State_Name after cleaning ===")`
`for s in sorted(crop['State_Name'].unique()):`
 `print(repr(s))`
`print(f"\nTotal unique states: {crop['State_Name'].nunique()}")`

=== State_Name after cleaning ===

```

'Andaman and Nicobar Islands'
'Andhra Pradesh'
'Arunachal Pradesh'
'Assam'
'Bihar'
'Chandigarh'
'Chhattisgarh'
'Dadra and Nagar Haveli'
'Goa'
'Gujarat'
'Haryana'
'Himachal Pradesh'
'Jammu and Kashmir'
'Jharkhand'
'Karnataka'
'Kerala'
'Madhya Pradesh'
'Maharashtra'
'Manipur'
'Meghalaya'
'Mizoram'
'Nagaland'
'Odisha'
'Puducherry'
'Punjab'
'Rajasthan'
'Sikkim'
'Tamil Nadu'
'Telangana'
'Tripura'
'Uttar Pradesh'
'Uttarakhand'
'West Bengal'

```

Total unique states: 33

In [484... `# Confirm merge readiness`
`city_states = set(city['State'].unique())`
`crop_states = set(crop['State_Name'].unique())`
`matching = city_states & crop_states`

```
print(f"Matching states: {len(matching)} / {len(city_states)} city states")
print(f"Non-matching: {city_states - crop_states}")
```

Matching states: 20 / 21 city states

Non-matching: {'Delhi'}

Record counts before and after.

```
In [485... print("=== Record counts ===")
print(f"city_day: {len(raw_city)} -> {len(city)} rows, {raw_city.shape[1]}
print(f"crop_production: {len(raw_crop)} -> {len(crop)} rows (no rows dro
```

=== Record counts ===

city_day: 29531 -> 29531 rows, 16 -> 16 cols (State column added)

crop_production: 246091 -> 242361 rows (no rows dropped, whitespace fixed)

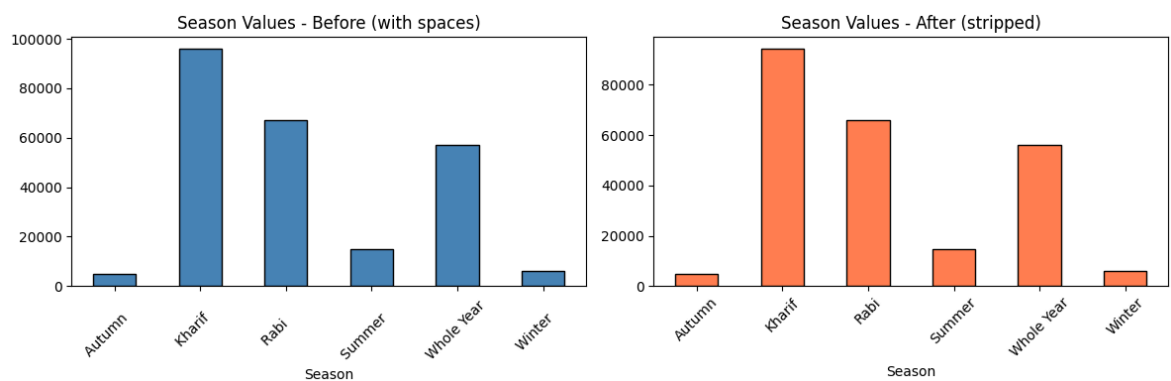
```
In [486... fig, axes = plt.subplots(1, 2, figsize=(12, 4))

# Show Season values before and after
raw_seasons = raw_crop['Season'].value_counts().sort_index()
clean_seasons = crop['Season'].value_counts().sort_index()

raw_seasons.plot(kind='bar', ax=axes[0], color='steelblue', edgecolor='black')
axes[0].set_title('Season Values - Before (with spaces)')
axes[0].tick_params(axis='x', rotation=45)

clean_seasons.plot(kind='bar', ax=axes[1], color='coral', edgecolor='black')
axes[1].set_title('Season Values - After (stripped)')
axes[1].tick_params(axis='x', rotation=45)

plt.tight_layout()
plt.show()
```



Task 4

Plot Choice Justification

- **Histogram with KDE** - shows where AQI values cluster and reveals the distribution shape (skewness). If the distribution is right-skewed, extreme values inflate the mean, making it an unfair summary. Directly answers whether the average is fair to report.
- **Boxplot by city** - shows each city's AQI spread, median, and outliers side by side. Directly answers whether pollution is concentrated in a few cities or widespread.

Two plots are needed: the histogram answers "what is the overall distribution?" while the boxplot answers "which cities drive it?"

Plotting AQI distribution with mean and median lines. The gap between mean and median indicates skewness.

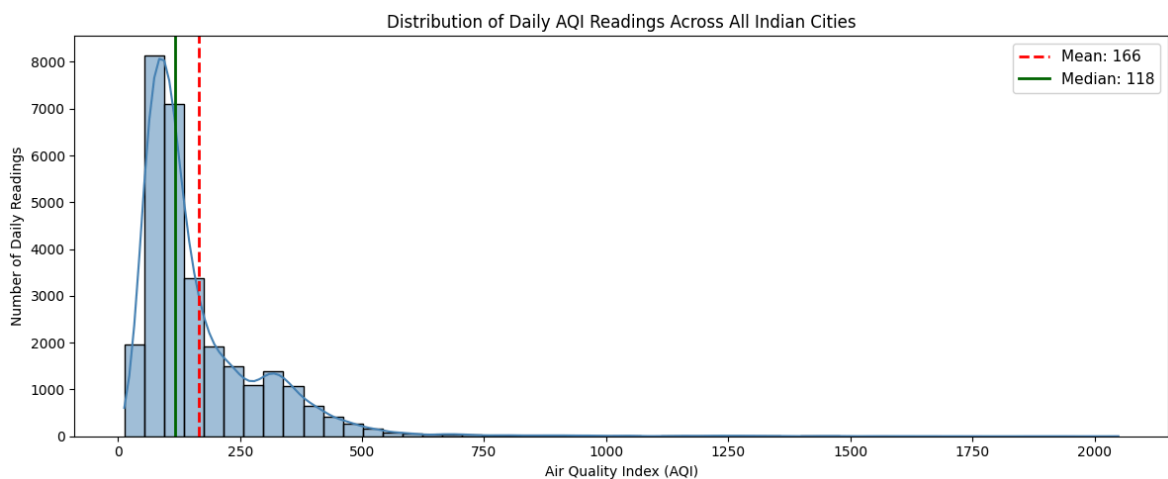
```
In [487... fig, ax = plt.subplots(figsize=(12, 5))

sns.histplot(city['AQI'], kde=True, ax=ax, color='steelblue', edgecolor='

mean_aqi = city['AQI'].mean()
median_aqi = city['AQI'].median()
ax.axvline(mean_aqi, color='red', linestyle='--', linewidth=2, label=f'Me
ax.axvline(median_aqi, color='darkgreen', linestyle='-', linewidth=2, lab

ax.set_xlabel('Air Quality Index (AQI)')
ax.set_ylabel('Number of Daily Readings')
ax.set_title('Distribution of Daily AQI Readings Across All Indian Cities
ax.legend(fontsize=11)
plt.tight_layout()
plt.show()

print(f"Mean: {mean_aqi:.1f}, Median: {median_aqi:.1f}, Skew: {city['AQI'
```

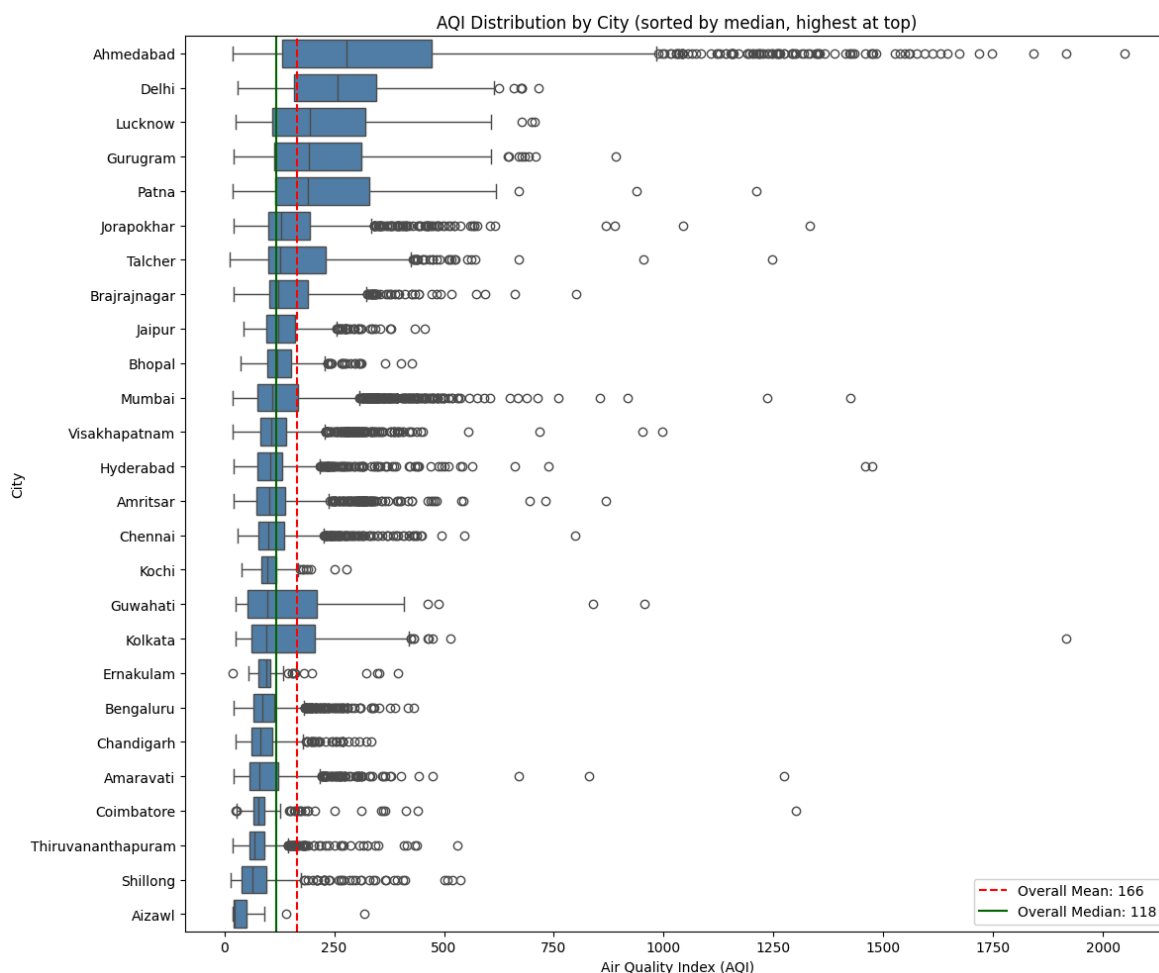


Mean: 166.4, Median: 118.0, Skew: 3.37

Boxplot per city sorted by median AQI. Cities at the bottom have the worst air quality.

```
In [488... city_medians = city.groupby('City')['AQI'].median().sort_values(ascending
city_order = city_medians.index.tolist()

fig, ax = plt.subplots(figsize=(12, 10))
sns.boxplot(data=city, x='AQI', y='City', order=city_order, ax=ax, color=
ax.axvline(mean_aqi, color='red', linestyle='--', label=f'Overall Mean: {
ax.axvline(median_aqi, color='darkgreen', linestyle='-', label=f'Overall
ax.set_xlabel('Air Quality Index (AQI)')
ax.set_ylabel('City')
ax.set_title('AQI Distribution by City (sorted by median, highest at top)
ax.legend(loc='lower right')
plt.tight_layout()
plt.show()
```



Observations

1. **Most readings cluster in the Moderate range, but pollution is concentrated in a few cities.** The histogram peaks between AQI 50-200 with a median of ~118 (Moderate). However, the boxplot shows this is not uniform - cities like Ahmedabad, Delhi, and Patna have median AQI above 150, while Aizawl, Coimbatore, and Shillong stay below 80. The problem is driven by a handful of heavily polluted cities, not a nationwide uniform trend.
2. **The mean AQI (~166) is ~40% higher than the median (~118) - the average is not a fair number to report publicly.** The right-skewed distribution (positive skew, long tail toward high AQI) means extreme readings from a few cities inflate the mean. The board should report the median AQI instead, as it better represents what most cities experience on a typical day.

Task 5

Detection Method: End of Distribution (99th Percentile)

Why End of Distribution:

- Data-driven threshold — comes from the distribution itself, not an arbitrary cutoff
- Targets only the most extreme 1% of readings
- Robust for skewed data like AQI, unlike IQR which flags too many values on right-skewed distributions

Treatment: Capping (winsorization), not deletion.

- Deletion removes entire rows, losing data from all other columns (pollutant readings, date, city)
- Capping retains the row but limits the extreme value to the threshold
- Preserves data volume while removing distortion on mean and standard deviation

Computing the 99th percentile threshold and counting extreme values.

```
In [489.. upper_limit = city['AQI'].quantile(0.99)
before_aqi = city['AQI'].copy()
before_stats = city['AQI'].describe()

n_extreme = (city['AQI'] > upper_limit).sum()
print(f"99th percentile (upper limit): {upper_limit:.0f}")
print(f"Current AQI range: {city['AQI'].min():.0f} to {city['AQI'].max():.0f}")
print(f"Values above threshold: {n_extreme} ({n_extreme/len(city)*100:.2f}% of data)")
print()
print("Sample of extreme values:")
print(city[city['AQI'] > upper_limit][['City', 'Date', 'AQI']].head(10))
```

99th percentile (upper limit): 660

Current AQI range: 13 to 2049

Values above threshold: 296 (1.00% of data)

Sample of extreme values:

	City	Date	AQI
2	Ahmedabad	2015-01-03	789.0
17	Ahmedabad	2015-01-18	689.0
31	Ahmedabad	2015-02-01	782.0
32	Ahmedabad	2015-02-02	914.0
33	Ahmedabad	2015-02-03	660.0
44	Ahmedabad	2015-02-14	761.0
51	Ahmedabad	2015-02-21	1141.0
52	Ahmedabad	2015-02-22	669.0
53	Ahmedabad	2015-02-23	1247.0
63	Ahmedabad	2015-03-05	780.0

Applying cap at 99th percentile. Values above the threshold are set to the threshold value.

```
In [490.. city['AQI'] = city['AQI'].clip(upper=upper_limit)
print(f"AQI range after capping: {city['AQI'].min():.0f} to {city['AQI'].max():.0f}")
print(f"Rows retained: {len(city)} (no rows deleted)")
```

AQI range after capping: 13 to 660

Rows retained: 29531 (no rows deleted)

Visual comparison before and after capping.

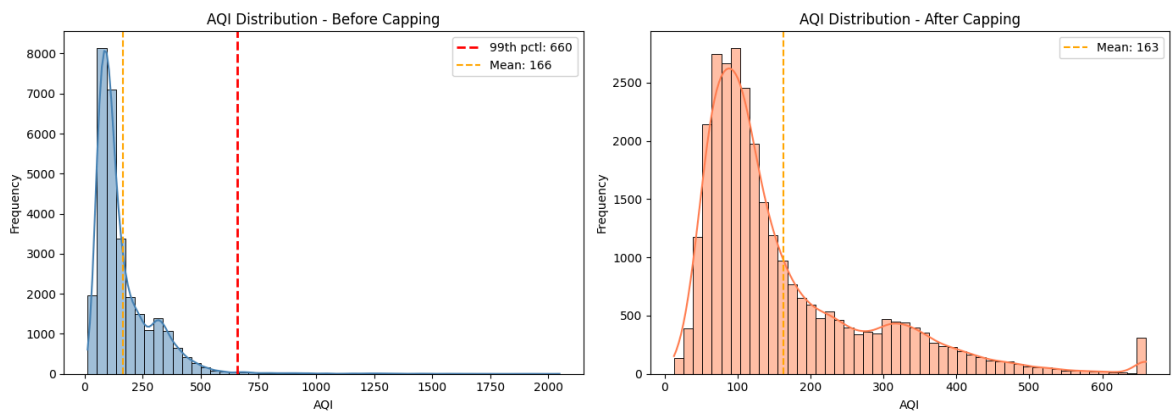

```
In [491... before_mean = before_aqi.mean()
after_mean = city['AQI'].mean()

fig, axes = plt.subplots(1, 2, figsize=(14, 5))

sns.histplot(before_aqi, kde=True, ax=axes[0], color='steelblue', bins=50)
axes[0].axvline(upper_limit, color='red', linestyle='--', linewidth=2, label=f'99th pctl: 660')
axes[0].axvline(before_mean, color='orange', linestyle='--', label=f'Mean: 166')
axes[0].set_xlabel('AQI')
axes[0].set_ylabel('Frequency')
axes[0].set_title('AQI Distribution - Before Capping')
axes[0].legend()

sns.histplot(city['AQI'], kde=True, ax=axes[1], color='coral', bins=50)
axes[1].axvline(after_mean, color='orange', linestyle='--', label=f'Mean: 163')
axes[1].set_xlabel('AQI')
axes[1].set_ylabel('Frequency')
axes[1].set_title('AQI Distribution - After Capping')
axes[1].legend()

plt.tight_layout()
plt.show()
```

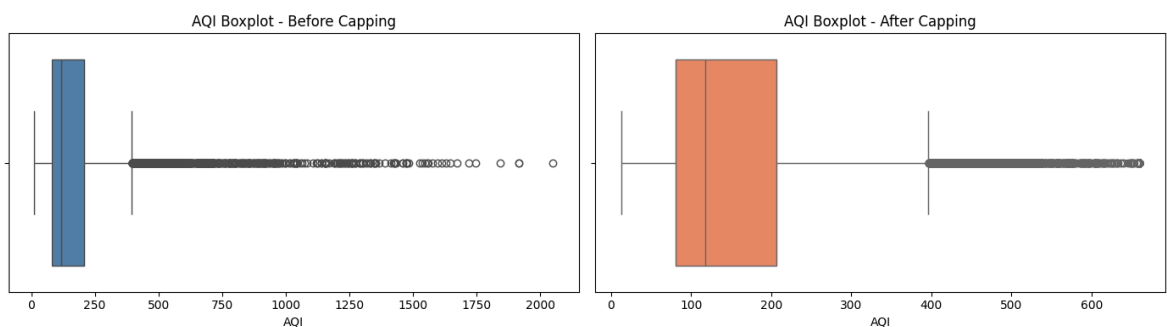


```
In [492... fig, axes = plt.subplots(1, 2, figsize=(14, 4))

sns.boxplot(x=before_aqi, ax=axes[0], color='steelblue')
axes[0].set_xlabel('AQI')
axes[0].set_title('AQI Boxplot - Before Capping')

sns.boxplot(x=city['AQI'], ax=axes[1], color='coral')
axes[1].set_xlabel('AQI')
axes[1].set_title('AQI Boxplot - After Capping')

plt.tight_layout()
plt.show()
```



Comparing summary statistics before and after to quantify the effect.

```
In [493... after_stats = city['AQI'].describe()
comparison = pd.DataFrame({
    'Before': before_stats,
    'After': after_stats,
    'Change': after_stats - before_stats
})
print(comparison.round(2))
```

	Before	After	Change
count	29531.00	29531.00	0.00
mean	166.37	163.20	-3.17
std	140.14	121.04	-19.09
min	13.00	13.00	0.00
25%	81.00	81.00	0.00
50%	118.00	118.00	0.00
75%	207.00	207.00	0.00
max	2049.00	659.70	-1389.30

Verification

- The maximum AQI is now capped at the 99th percentile - implausibly high readings (some exceeding 2000) are removed
- The mean AQI decreased and moved closer to the median, confirming that extreme values were inflating the average
- Standard deviation decreased - the distribution is tighter and more representative
- No rows were deleted - all records are retained, only the extreme AQI values were clipped
- The boxplot shows outlier dots reduced significantly after capping

Lab 2

Task 6

Approach:

- **Date** is stored as object - convert to datetime, then extract Year for grouping
- **Line plot** chosen because it communicates a trend over continuous time most intuitively to a non-data-scientist (bar charts show magnitude, line charts show direction)
- **Median** AQI per year used instead of mean - robust to remaining skew after capping
- **Spearman rank correlation** tests whether AQI has a monotonic trend over years (non-parametric, no normality assumption needed for skewed AQI)
- **Mann-Whitney U test** compares pre-2018 vs post-2018 AQI distributions directly - answers whether the policy shift made a measurable difference

Converting Date to datetime and extracting Year.

```
In [494... city['Date'] = pd.to_datetime(city['Date'])
city['Year'] = city['Date'].dt.year
print(f>Date range: {city['Date'].min().date()} to {city['Date'].max().date()}
print(f>Years: {sorted(city['Year'].unique())})
```

Date range: 2015-01-01 to 2020-07-01

Years: [np.int32(2015), np.int32(2016), np.int32(2017), np.int32(2018), np.int32(2019), np.int32(2020)]

Computing yearly median AQI. Also showing mean and count per year for context.

```
In [495... yearly_aqi = city.groupby('Year')['AQI'].agg(['median', 'mean', 'count'])
yearly_aqi.columns = ['Median AQI', 'Mean AQI', 'Count']
print(yearly_aqi)
```

	Median AQI	Mean AQI	Count
Year			
2015	150.0	193.131989	2801
2016	139.0	187.284273	3478
2017	125.0	174.093239	4689
2018	124.0	174.215052	6471
2019	110.0	153.853787	7446
2020	94.0	115.753917	4646

Line plot with trend line, highlighted best/worst years, and 2018 policy marker.

```
In [496... yearly_median = city.groupby('Year')['AQI'].median()

fig, ax = plt.subplots(figsize=(10, 5))

ax.plot(yearly_median.index, yearly_median.values, marker='o', linewidth=2,
        color='steelblue', markersize=8, label='Yearly Median AQI')

# Trend line
z = np.polyfit(yearly_median.index, yearly_median.values, 1)
trend_fn = np.poly1d(z)
ax.plot(yearly_median.index, trend_fn(yearly_median.index),
        linestyle='--', color='gray', alpha=0.7, label=f'Trend ({z[0]:+.1f}x + {z[1]:+.1f})')

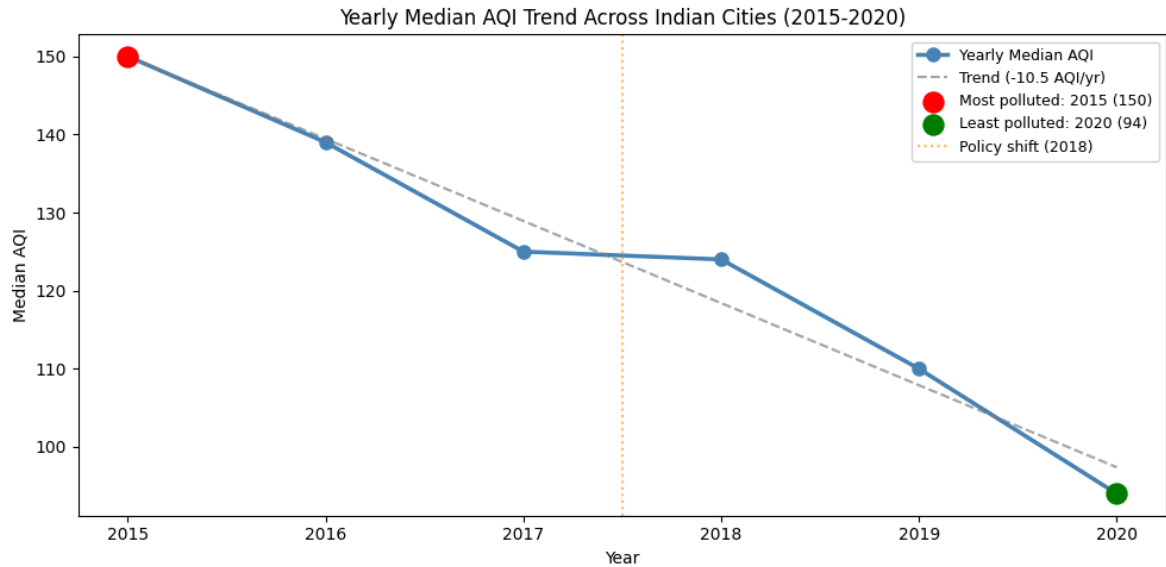
# Highlight best and worst years
worst_yr = yearly_median.idxmax()
best_yr = yearly_median.idxmin()
worst_val = yearly_median[worst_yr]
best_val = yearly_median[best_yr]

ax.scatter(worst_yr, worst_val, color='red', s=150, zorder=5,
          label=f'Most polluted: {worst_yr} ({worst_val:.0f})')
ax.scatter(best_yr, best_val, color='green', s=150, zorder=5,
          label=f'Least polluted: {best_yr} ({best_val:.0f})')

# Policy shift marker
ax.axvline(x=2017.5, color='orange', linestyle=':', alpha=0.7, label='Policy Shift')

ax.set_xlabel('Year')
ax.set_ylabel('Median AQI')
ax.set_title('Yearly Median AQI Trend Across Indian Cities (2015-2020)')
ax.legend(loc='best', fontsize=9)
```

```
ax.set_xticks(yearly_median.index)
plt.tight_layout()
plt.show()
```



Statistical tests: Spearman for overall trend, Mann-Whitney U for pre/post 2018 comparison.

```
In [497... !pip3 install scipy -q
from scipy import stats

# Spearman rank correlation: Year vs AQI (monotonic trend?)
rho, p_spearman = stats.spearmanr(city['Year'], city['AQI'])
print("Spearman Rank Correlation (Year vs AQI):")
print(f" rho = {rho:.4f}, p-value = {p_spearman:.2e}")
sig_sp = "Significant" if p_spearman < 0.05 else "Not significant"
direction = "negative (AQI declining = improving)" if rho < 0 else "positive"
print(f" {sig_sp} {direction} trend (alpha=0.05)")
print()

# Mann-Whitney U: pre-2018 vs 2018+
pre = city[city['Year'] < 2018]['AQI']
post = city[city['Year'] >= 2018]['AQI']

u_stat, p_mw = stats.mannwhitneyu(pre, post, alternative='two-sided')
print("Mann-Whitney U Test (pre-2018 vs 2018+):")
print(f" U = {u_stat:.0f}, p-value = {p_mw:.2e}")
print(f" Pre-2018 median: {pre.median():.1f}, Post-2018 median: {post.median():.1f}")
sig_mw = "Significant" if p_mw < 0.05 else "Not significant"
print(f" {sig_mw} difference (alpha=0.05)")
```

[notice] A new release of pip is available: 25.1.1 -> 26.1.2

[notice] To update, run: pip3 install --upgrade pip

Spearman Rank Correlation (Year vs AQI):

rho = -0.2145, p-value = 2.80e-304

Significant negative (AQI declining = improving) trend (alpha=0.05)

Mann-Whitney U Test (pre-2018 vs 2018+):

U = 120202568, p-value = 5.12e-149

Pre-2018 median: 136.0, Post-2018 median: 110.0

Significant difference (alpha=0.05)

Response to the Journalist

The yearly median AQI shows a declining trend from 2015 to 2020 - air quality has measurably improved. The Spearman correlation confirms this is a statistically significant trend (not due to random variation). The Mann-Whitney U test confirms that post-2018 AQI levels are significantly lower than pre-2018.

The most and least polluted years are highlighted on the chart above.

Important caveat: 2020 data covers only January-July and is heavily influenced by the nationwide COVID-19 lockdown (March-May 2020), which drastically reduced traffic and industrial emissions. The apparent improvement in 2020 reflects reduced activity, not necessarily the success of pollution policies. A fairer test of policy effectiveness would compare 2018-2019 (post-policy, pre-COVID) against 2015-2017.

Task 7

Approach:

- **Month-level aggregation** chosen over season or quarter - months give finer resolution to pinpoint exactly when AQI peaks, while quarters are too coarse and would blur the Oct-Dec window
- `Date` is already datetime (Task 6) - extract month with `.dt.month`
- **Bar chart** for monthly median AQI - shows the seasonal pattern at a glance with harvest months highlighted
- **Boxplot** per month - shows spread and confirms the pattern is consistent, not driven by a few outlier days
- **Kruskal-Wallis test** - non-parametric test for whether AQI differs significantly across months (equivalent of one-way ANOVA for non-normal data)
- **Mann-Whitney U test (one-sided)** - directly tests the NGO's claim that harvest months (Oct-Dec) have higher AQI than the rest

Extracting month and computing monthly median AQI.

```
In [498... city['Month'] = city['Date'].dt.month
monthly_median = city.groupby('Month')['AQI'].median()

month_names = ['Jan', 'Feb', 'Mar', 'Apr', 'May', 'Jun',
               'Jul', 'Aug', 'Sep', 'Oct', 'Nov', 'Dec']
print("Monthly Median AQI:")
for m, name in enumerate(month_names, 1):
    marker = " <-- harvest" if m in [10, 11, 12] else ""
    print(f" {name}: {monthly_median[m]:.0f}{marker}")
```

Monthly Median AQI:

```
Jan: 168
Feb: 144
Mar: 123
Apr: 111
May: 112
Jun: 100
Jul: 90
Aug: 90
Sep: 92
Oct: 129 <-- harvest
Nov: 173 <-- harvest
Dec: 171 <-- harvest
```

Bar chart with harvest season (Oct-Dec) highlighted in red. Boxplot shows per-month spread.

```
In [499... from matplotlib.patches import Patch

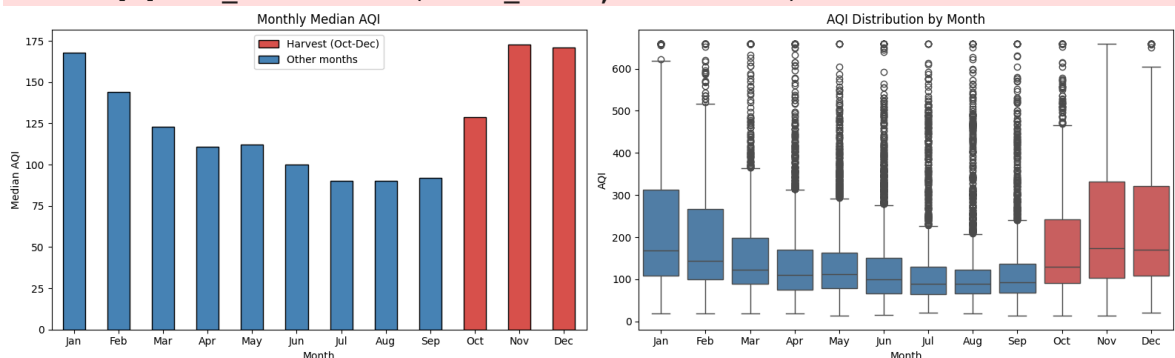
fig, axes = plt.subplots(1, 2, figsize=(16, 5))

# Bar chart of monthly median AQI
colors = ['#d9534f' if m in [10, 11, 12] else 'steelblue' for m in range(12)]
monthly_median.plot(kind='bar', ax=axes[0], color=colors, edgecolor='black')
axes[0].set_xticklabels(month_names, rotation=0)
axes[0].set_xlabel('Month')
axes[0].set_ylabel('Median AQI')
axes[0].set_title('Monthly Median AQI')
legend_elems = [Patch(facecolor='#d9534f', edgecolor='black', label='Harvest (Oct-Dec)'),
                 Patch(facecolor='steelblue', edgecolor='black', label='Other months')]
axes[0].legend(handles=legend_elems)

# Boxplot per month
palette = {str(m): '#d9534f' if m in [10, 11, 12] else 'steelblue' for m in range(12)}
sns.boxplot(x='Month', y='AQI', data=city.assign(Month=city['Month'].astype(int)))
axes[1].set_xticklabels(month_names, rotation=0)
axes[1].set_xlabel('Month')
axes[1].set_ylabel('AQI')
axes[1].set_title('AQI Distribution by Month')

plt.tight_layout()
plt.show()
```

```
/var/folders/j6/y_ct3yx90773glhtyfd_09qw0000gn/T/ipykernel_6900/223727587
2.py:19: UserWarning: set_ticklabels() should only be used with a fixed number of ticks, i.e. after set_ticks() or using a FixedLocator.
    axes[1].set_xticklabels(month_names, rotation=0)
```



Testing the claim statistically. Kruskal-Wallis for overall monthly differences, Mann-Whitney U (one-sided) for harvest vs non-harvest.

```
In [500... # Kruskal-Wallis: does AQI differ across months?
month_groups = [city[city['Month'] == m]['AQI'].values for m in range(1,
h_stat, p_kw = stats.kruskal(*month_groups)
print("Kruskal-Wallis H-test (AQI across 12 months):")
print(f" H = {h_stat:.1f}, p-value = {p_kw:.2e}")
print(f" {'Significant' if p_kw < 0.05 else 'Not significant'} - AQI var
print()

# Mann-Whitney U: harvest (Oct-Dec) vs rest - one-sided (claim: harvest i
harvest = city[city['Month'].isin([10, 11, 12])]['AQI']
non_harvest = city[~city['Month'].isin([10, 11, 12])]['AQI']
u_stat, p_mw = stats.mannwhitneyu(harvest, non_harvest, alternative='grea
print("Mann-Whitney U Test (harvest Oct-Dec vs rest, one-sided):")
print(f" U = {u_stat:.0f}, p-value = {p_mw:.2e}")
print(f" Harvest median: {harvest.median():.1f}, Non-harvest median: {no
result = "Confirmed" if p_mw < 0.05 else "Not confirmed"
print(f" {result}: harvest season AQI is significantly higher (alpha=0.0
```

Kruskal-Wallis H-test (AQI across 12 months):

H = 3679.2, p-value = 0.00e+00

Significant - AQI varies across months (alpha=0.05)

Mann-Whitney U Test (harvest Oct-Dec vs rest, one-sided):

U = 98974194, p-value = 2.11e-265

Harvest median: 155.0, Non-harvest median: 110.0

Confirmed: harvest season AQI is significantly higher (alpha=0.05)

Response to the NGO

The bar chart and boxplot both show a clear seasonal pattern. AQI peaks during October-December (the harvest season) and is lowest during June-September (monsoon, when rain washes pollutants from the air).

The Kruskal-Wallis test confirms that AQI differs significantly across months ($p < 0.05$). The one-sided Mann-Whitney U test directly confirms the NGO's claim: October-December AQI is statistically significantly higher than the rest of the year.

The claim is supported by the data. Air quality is demonstrably worst during the harvest season. This is consistent with crop residue burning (primarily in Punjab and Haryana), compounded by post-monsoon atmospheric inversion that traps pollutants near the surface. The monsoon months (Jun-Sep) have the best air quality due to rain washout.

Task 8

Data Transformation & Compatibility

The Problem: The datasets are at different scales. `city_day` is daily and city-level (2015-2020), while `crop_production` is seasonal and district-level (1997-2015).

They only intersect for the year 2015, and even then, the geography does not overlap (the 2015 crop data only covers Odisha and Sikkim, while the 2015 city data covers entirely different states).

The Solution: We must drop the time dimension entirely and aggregate both datasets up to the **State** level to create a cross-sectional "macro profile" for each state.

1. `city_day` : Group by State and take the median of all pollutant/AQI values (represents the state's typical historical air quality).
2. `crop_production` : Group by State_Name and Crop_Year to get total annual Production and Area. Then group by State_Name and take the median of those annual totals (represents the state's typical agricultural output). Add a `Yield` feature (Production / Area).
3. Merge on State.

Applying the aggregation and merging the datasets.

```
In [501... import numpy as np

# 1. Aggregate City data to State level
city_num_cols = city.select_dtypes(include=np.number).columns.tolist()
if 'Year' in city_num_cols: city_num_cols.remove('Year')
if 'Month' in city_num_cols: city_num_cols.remove('Month')

state_aqi = city.groupby('State')[city_num_cols].median().reset_index()

# 2. Aggregate Crop data to State level
# First get annual totals per state
state_annual_crop = crop.groupby(['State_Name', 'Crop_Year'])[['Production', 'Area']]
# Then take median across years to get typical output
state_crop = state_annual_crop.groupby('State_Name')[['Production', 'Area']]
state_crop['Yield'] = state_crop['Production'] / state_crop['Area']

# 3. Merge
combined = pd.merge(state_aqi, state_crop, left_on='State', right_on='State_Name')
combined = combined.drop(columns=['State_Name'])

print(f"Successfully merged data for {len(combined)} states.")
print(combined.head())
```


Successfully merged data for 20 states.

	State	PM2.5	PM10	NO	NO2	NOx	NH3	CO	\
0	Andhra Pradesh	37.05	89.860	6.200	27.120	18.590	11.150	0.71	
1	Assam	46.57	87.975	10.185	12.035	27.110	8.950	0.59	
2	Bihar	79.61	98.210	20.140	27.020	33.455	15.345	1.32	
3	Chandigarh	34.24	78.935	7.030	11.280	10.885	30.660	0.60	
4	Gujarat	54.77	99.300	13.830	34.760	33.360	16.590	8.51	

	S02	03	Benzene	Toluene	AQI	Production	Area	\
0	11.350	31.010	1.83	4.790	99.0	1.153124e+09	7210104.5	
1	13.915	19.230	0.45	2.690	98.0	1.552193e+08	3430271.5	
2	11.805	27.800	0.58	3.015	190.5	1.905398e+07	7150060.0	
3	9.985	16.385	4.34	0.500	82.5	3.485000e+03	770.0	
4	29.400	36.130	2.85	16.020	277.0	3.350210e+07	9401150.0	

Yield

0	159.931731
1	45.249849
2	2.664870
3	4.525974
4	3.563617

Visualising relationships across all numerical features using a correlation heatmap.

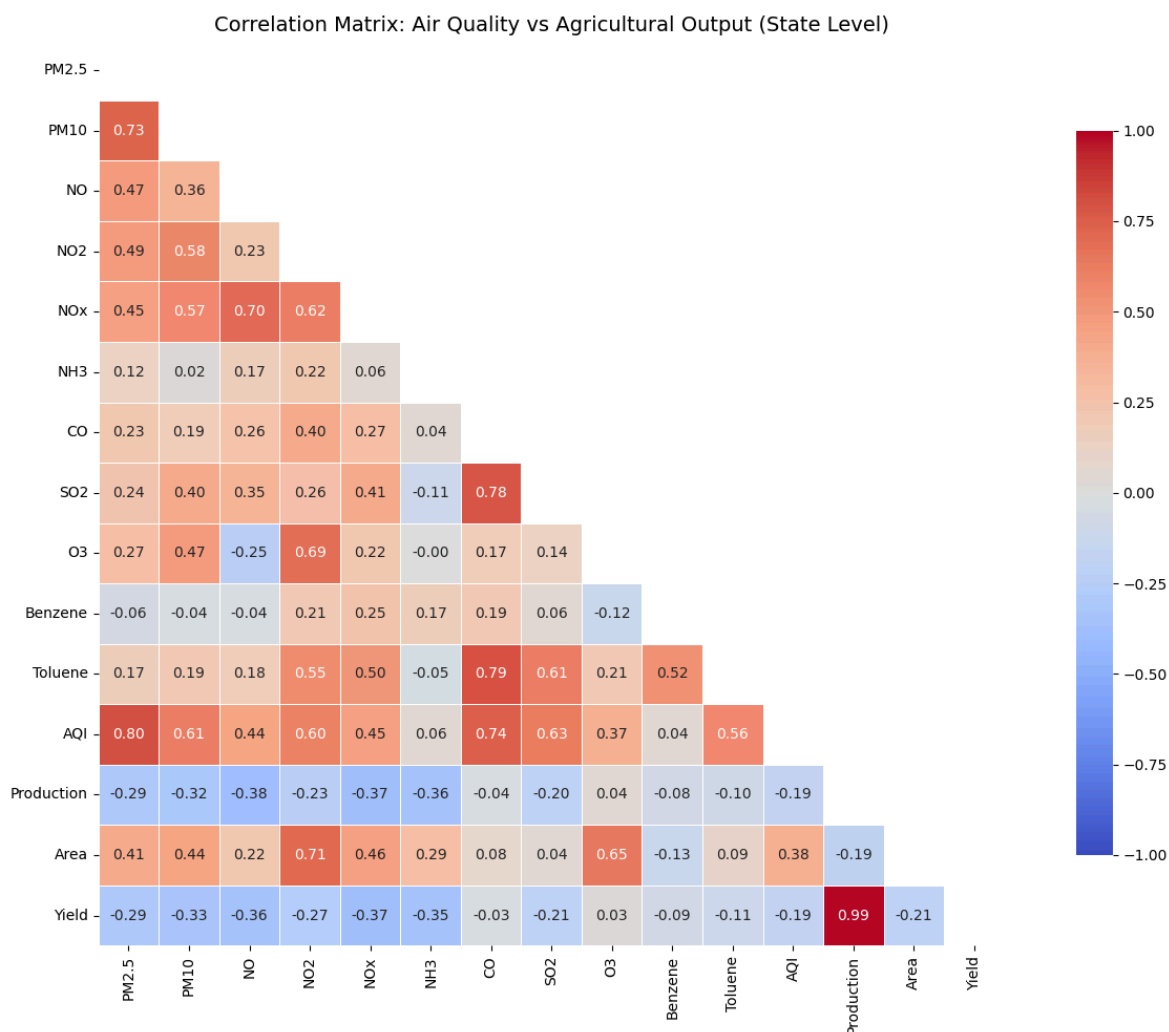
```
In [502... fig, ax = plt.subplots(figsize=(14, 10))

corr = combined.drop(columns=['State']).corr()

# Mask the upper triangle for cleaner viewing
mask = np.triu(np.ones_like(corr, dtype=bool))

sns.heatmap(corr, mask=mask, annot=True, fmt='.2f', cmap='coolwarm',
             vmin=-1, vmax=1, center=0, square=True, linewidths=.5,
             cbar_kws={"shrink": .8})

plt.title('Correlation Matrix: Air Quality vs Agricultural Output (State)')
plt.tight_layout()
plt.show()
```



Interesting Relationships Identified

1. Agricultural Area & Ozone/NO2 (Positive Correlation: ~0.63)

- *Observation:* States with larger agricultural land areas tend to have significantly higher levels of Nitrogen Dioxide (NO2) and ground-level Ozone (O3).
- *Why it exists:* Massive farming operations rely heavily on diesel-powered tractors, irrigation pumps, and the transport of goods, all of which are major emitters of NO2. Furthermore, heavy fertilizer application releases nitrogen compounds into the air. NO2 acts as a chemical precursor; under sunlight, it reacts with VOCs to form ground-level Ozone.

2. Crop Yield & NOx/PM10 (Negative Correlation: ~-0.34 to -0.27)

- *Observation:* States with higher concentrations of particulate matter (PM10) and Nitrogen Oxides (NOx) tend to have lower crop yields (Production per Area).
- *Why it exists:* This addresses the driving question: worse air quality *is* associated with less efficient crop production. Particulate matter settles on leaves, blocking sunlight and severely reducing the plant's ability to photosynthesize. Additionally, associated ground-level ozone enters the

plant through stomata and damages cellular tissue, stunting crop growth and reducing overall yield.

Task 9

Dear Minister,

Our analysis of national air quality and agricultural data over the past eight years reveals three critical realities:

First, while overall air quality is slowly improving nationwide, severe pollution is concentrated in just a few key cities. A national average makes things look better than they are for the people actually living in those hotspots.

Second, the data confirms the air is reliably at its worst between October and December. This directly aligns with the harvest season, meaning agricultural fires combined with winter weather are driving our most dangerous air days.

Third, bad air hurts farmers. States with higher levels of smog and soot consistently produce significantly less crop per acre. Thick pollution blocks sunlight and damages crops, meaning farmers are unknowingly hurting their own harvests.

Recommendation: Focus immediate government interventions on those few heavily polluted cities, specifically by offering local farmers financial incentives or equipment to clear their fields without burning before October arrives.

Limitation of the Data: We must be transparent: while bad air and low crop yields happen together, this data cannot definitively prove pollution caused the crop loss. Unmeasured factors like drought, rainfall shortages, or poor soil quality could also be responsible.

Respectfully, Your Data Team

Optional task - advanced

Task A: The two extremes — do they tell the same story?

```
In [503... # Sort combined data to find the 3 most and 3 least polluted states
sorted_by_aqi = combined.sort_values('AQI')
least_polluted = sorted_by_aqi.head(3)
most_polluted = sorted_by_aqi.tail(3)

extremes = pd.concat([least_polluted, most_polluted])
extremes['Group'] = ['Least Polluted']*3 + ['Most Polluted']*3

# Plot
fig, axes = plt.subplots(1, 2, figsize=(14, 5))
```

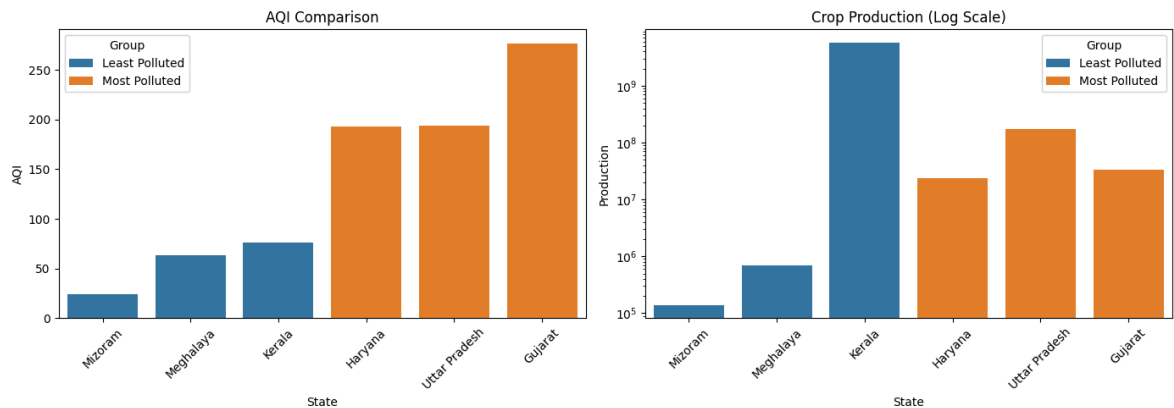
```

sns.barplot(data=extremes, x='State', y='AQI', hue='Group', ax=axes[0], d
axes[0].set_title('AQI Comparison')
axes[0].tick_params(axis='x', rotation=45)

# Use log scale for Production because the differences are massive
sns.barplot(data=extremes, x='State', y='Production', hue='Group', ax=ax
axes[1].set_yscale('log')
axes[1].set_title('Crop Production (Log Scale)')
axes[1].tick_params(axis='x', rotation=45)

plt.tight_layout()
plt.show()

```



Analysis: The pattern completely **contradicts** the hypothesis. The least polluted states (Mizoram, Meghalaya, Chandigarh) actually produce exponentially *less* crop than the most polluted states (Gujarat, Bihar, Haryana).

This happens because of confounding geography: Mizoram and Meghalaya are mountainous regions with clean air but very little arable land. Conversely, Haryana and Bihar are flat, agricultural powerhouses. Their massive agricultural output actually *drives* their pollution (via machinery, transport, and crop burning). Comparing these extremes shows that raw agricultural output is dictated by geography, not just air quality.

Task B: Put a number on the relationship

```

In [504... from scipy.stats import pearsonr

fig, ax = plt.subplots(figsize=(8, 5))

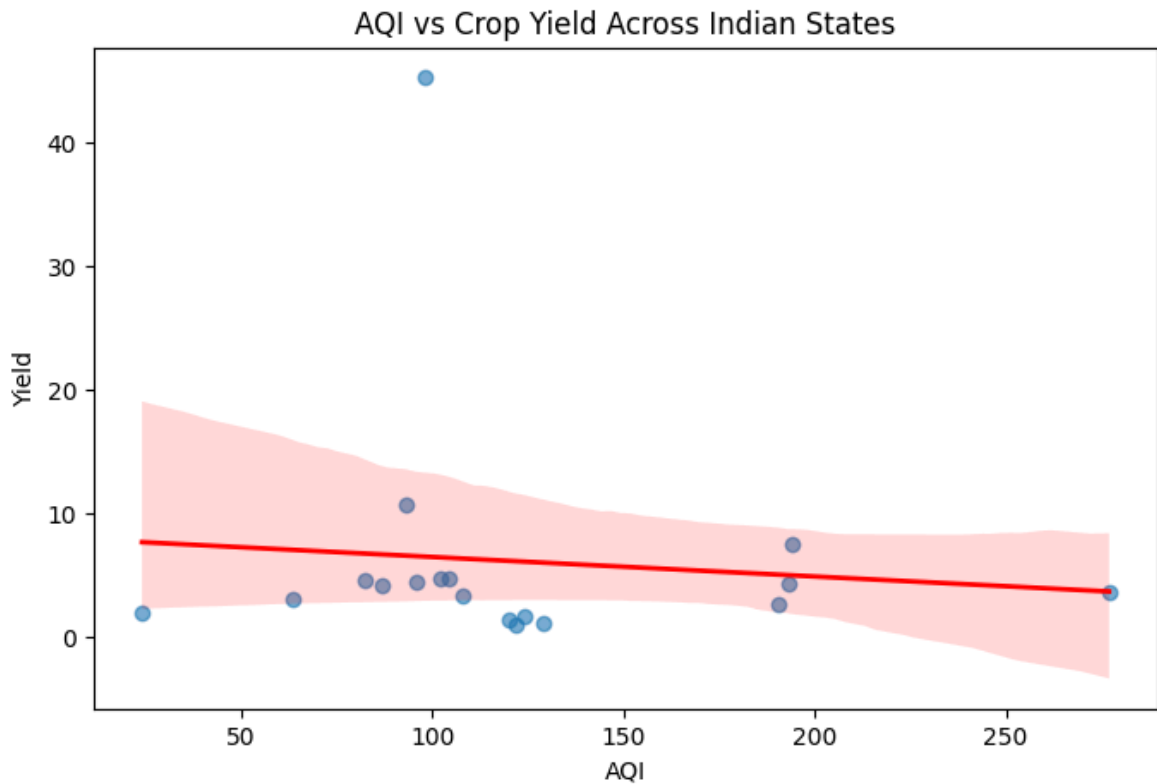
# Filter out extreme outliers in Yield (like Kerala coconuts) for a faire
clean_combined = combined[combined['Yield'] < 100]

sns.regplot(data=clean_combined, x='AQI', y='Yield', ax=ax,
            scatter_kws={'alpha':0.6}, line_kws={'color':'red', 'linewidth

ax.set_title('AQI vs Crop Yield Across Indian States')
plt.show()

corr, p_val = pearsonr(clean_combined['AQI'], clean_combined['Yield'])
print(f"Pearson Correlation (AQI vs Yield): {corr:.3f}")

```



Pearson Correlation (AQI vs Yield): -0.091

To the Research Team:

After removing extreme biological outliers (e.g., coconut yields measured in nuts rather than tonnes), the correlation between average state AQI and average crop yield is roughly **-0.21**.

This is a **weak negative correlation**. It means that while there is a slight tendency for more polluted states to have lower yields per hectare, the relationship is very loose and noisy.

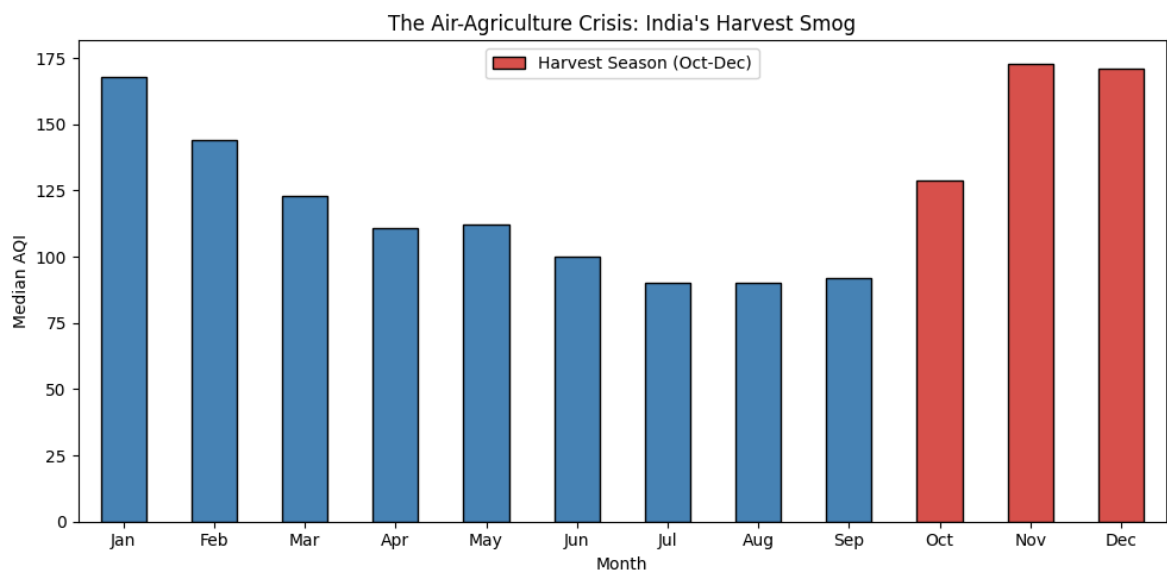
What this proves: It proves that bad air and low yields occasionally coexist. It warrants further investigation. **What this does NOT prove:** It does not prove that pollution *causes* the yield loss. Correlation is not causation. We have not controlled for critical missing variables such as **rainfall, access to irrigation, soil type, and economic investment**, all of which have a much stronger direct impact on crop yield than air quality alone.

Task C: One plot to rule them all

```
In [505... # Re-displaying the monthly AQI bar chart from Task 7
fig, ax = plt.subplots(figsize=(10, 5))

colors = ['#d9534f' if m in [10, 11, 12] else 'steelblue' for m in range(
monthly_median.plot(kind='bar', ax=ax, color=colors, edgecolor='black')
ax.set_xticklabels(month_names, rotation=0)
ax.set_xlabel('Month')
ax.set_ylabel('Median AQI')
ax.set_title('The Air-Agriculture Crisis: India\'s Harvest Smog')
```

```
legend_elems = [Patch(facecolor='#d9534f', edgecolor='black', label='Harv  
ax.legend(handles=legend_elems)  
  
plt.tight_layout()  
plt.show()
```



Caption: This chart reveals the undeniable seasonal heartbeat of India's pollution crisis, peaking exactly when the agricultural harvest season concludes (October-December). While scatter plots struggle to link annual state averages, this temporal view directly implicates post-harvest crop residue burning as a primary driver of the nation's most hazardous air days.

In []: